

Neural Network Implementation from Scratch

Divyansh Pandey

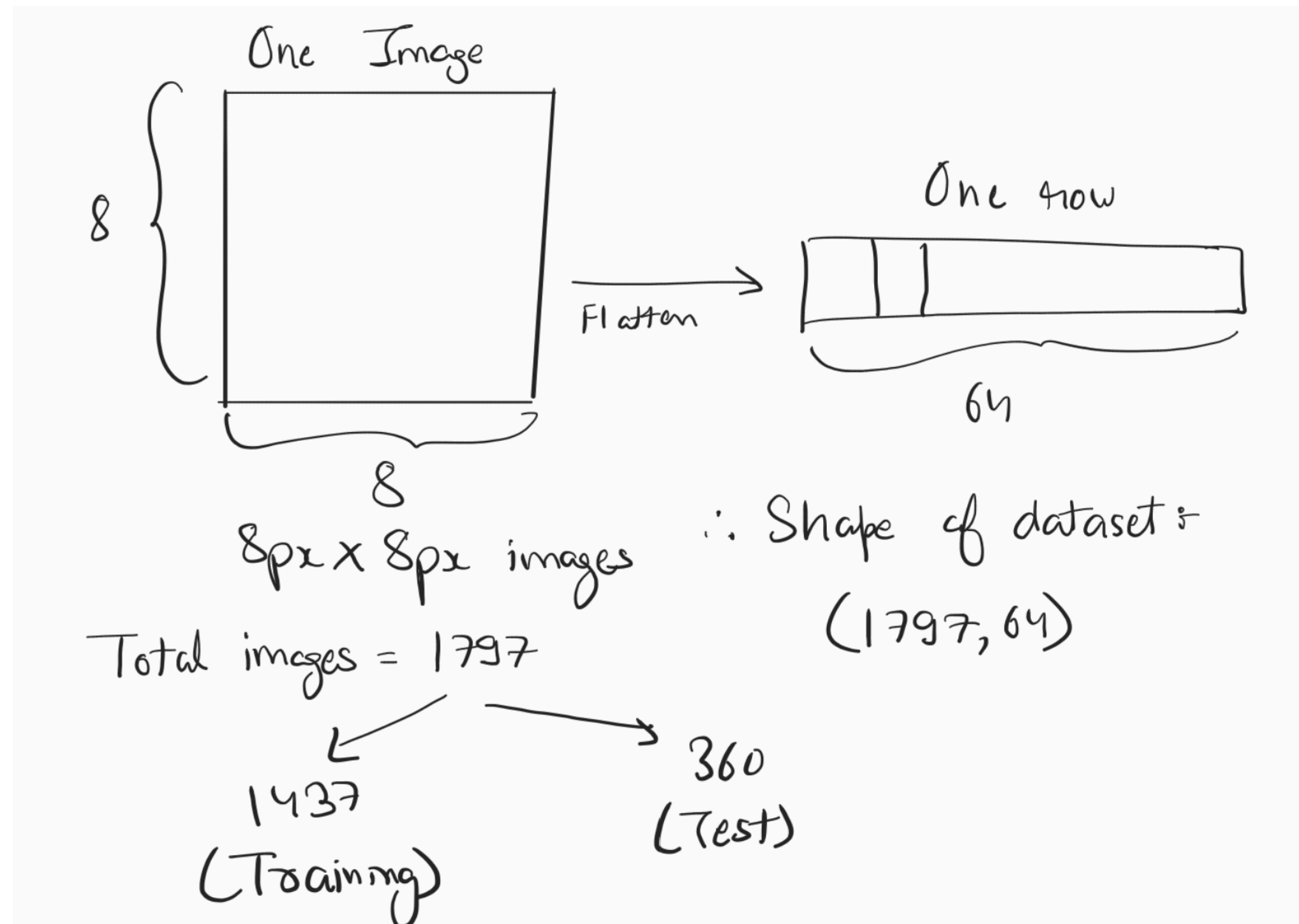
Dataset Structure and Input Shape

`sklearn.datasets.load_digits`

total images = 1797

With a 80-20 train-test split,
there will be 1437 images
in training set and 360 in test set

Input dims: (1797, 64)



Network Architecture

Input $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 10

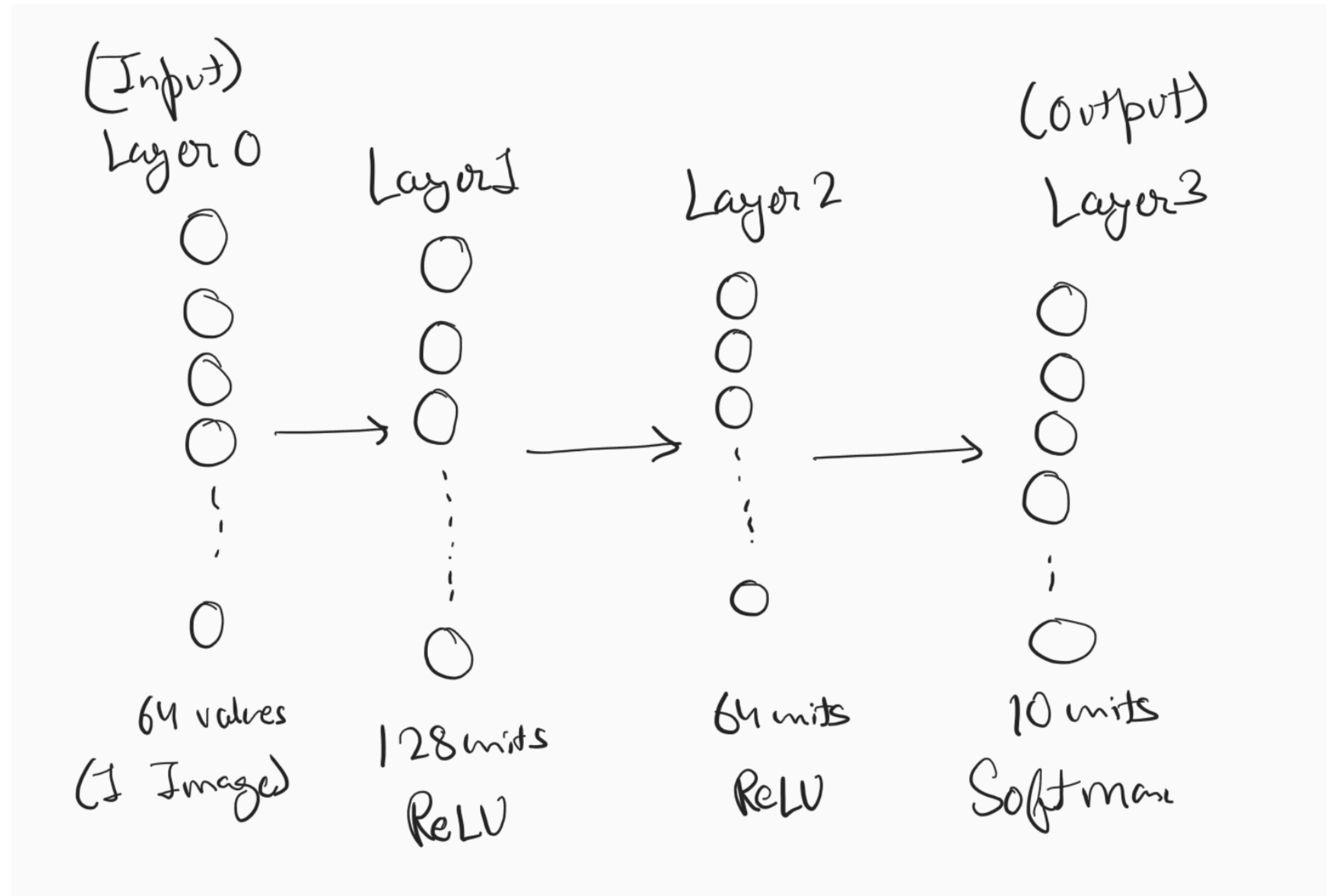
3-layer MLP

Layer 0 $\rightarrow$ Input

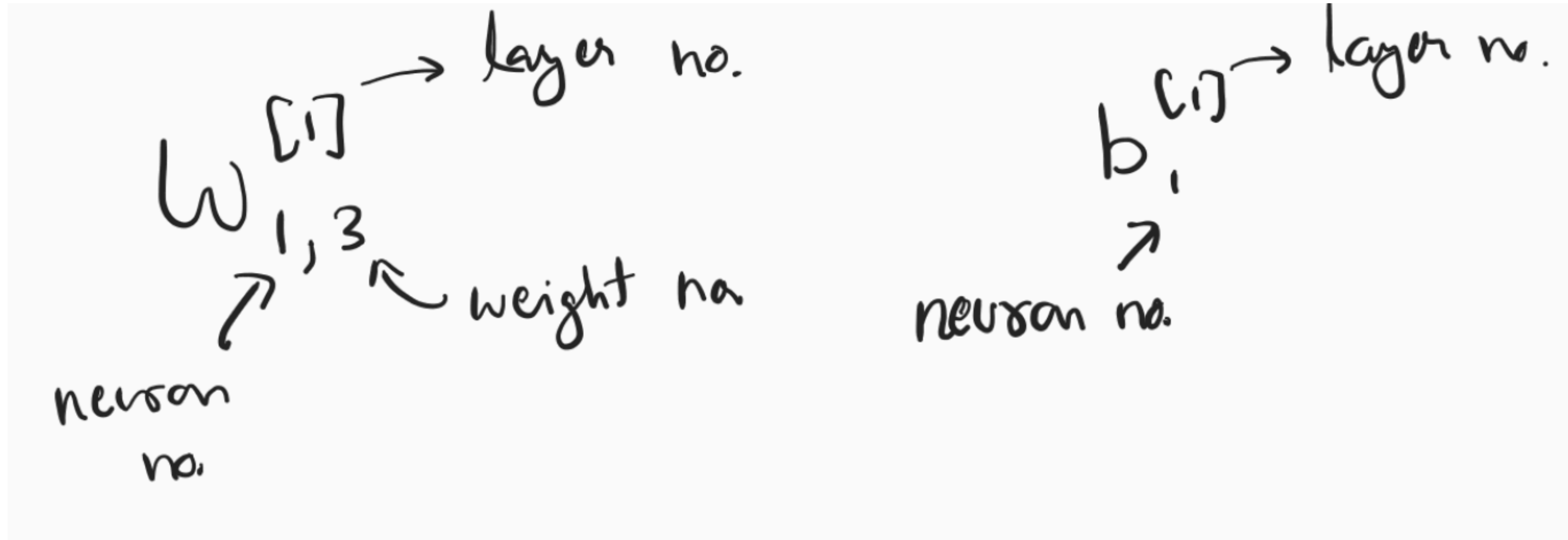
Layer 1 $\rightarrow$ 128 units (ReLU)

Layer 2 $\rightarrow$ 64 units (ReLU)

Layer 3 $\rightarrow$ 10 units (Softmax)
(Output)



Convention used



Weights Initialization (Kaiming Initialization)

To solve the problem of exploding or vanishing gradients in a neural network, we use different kinds of weights initialization which tends to keep the variance of output of each layer in check by initializing the weights in such a way that the variance of 'Z' for each layer remains constant and Z neither explodes or vanishes. In case when we use ReLU activation, we multiply a '2' with the variance, because ReLU on average zeros out half of the inputs, which further pushes down the variance by a factor of two. To compensate this, the 2 is multiplied. Without this 2, it is called 'Xavier's initialization'.

To achieve that, we choose the weights randomly from standard normal distribution and product them with required variance, which helps us to choose weights with specific variance.

$$W \sim \mathcal{N}(0, 1) \times \sqrt{\frac{2}{n_{\text{in}}}}$$

```
units_l1 = 128
units_l2 = 64
units_l3 = 10

# Initializing the values of weights and biases
w1 = np.random.randn(X_train.shape[1], units_l1) * np.sqrt(2/X_train.shape[1])
b1 = np.zeros((1, units_l1))

w2 = np.random.randn(units_l1, units_l2) * np.sqrt(2/units_l1)
b2 = np.zeros((1, units_l2))

w3 = np.random.randn(units_l2, units_l3) * np.sqrt(2/units_l2)
b3 = np.zeros((1, units_l3))
```

Visualizing Weights and Biases for each unit (neuron)

In the example of Layer 1:

Each neuron gets 64 values
(one image)

all those values are multiplied
with weights and a bias
is added

The output of each neuron is
represented by z , and the matrix
of all values of z is 'Z1'

Layer 1

$$\bigcirc \Rightarrow z_1^{[1]} = w_{1,1}^{[1]} x_1 + w_{1,2}^{[1]} x_2 + \dots + w_{1,64}^{[1]} x_{64} + b_1^{[1]}$$

$$\bigcirc \Rightarrow z_2^{[1]} = w_{2,1}^{[1]} x_1 + w_{2,2}^{[1]} x_2 + \dots + w_{2,64}^{[1]} x_{64} + b_2^{[1]}$$

$\vdots$

$$\bigcirc \Rightarrow z_{128}^{[1]} = w_{128,1}^{[1]} x_1 + w_{128,2}^{[1]} x_2 + \dots + w_{128,64}^{[1]} x_{64} + b_{128}^{[1]}$$

128 units

ReLU

Activation function

In all the hidden layers,
ReLU is used as an activation
function

A1 will act as an activation
for layer 2 and is
calculated as: $A1 = \text{ReLU}(Z1)$

Since ReLU performs
element-wise operation, the
dimension of matrix do not change

$$\Rightarrow Z1 = [Z_1^{(1)} \ Z_2^{(1)} \ Z_3^{(1)} \ - \ - \ - \ - \ Z_{128}^{(1)}]$$

$$Z1 \Rightarrow (1, 128) \rightarrow \text{for "one image"}$$

$$\Rightarrow A1 = \text{ReLU}(Z1)$$

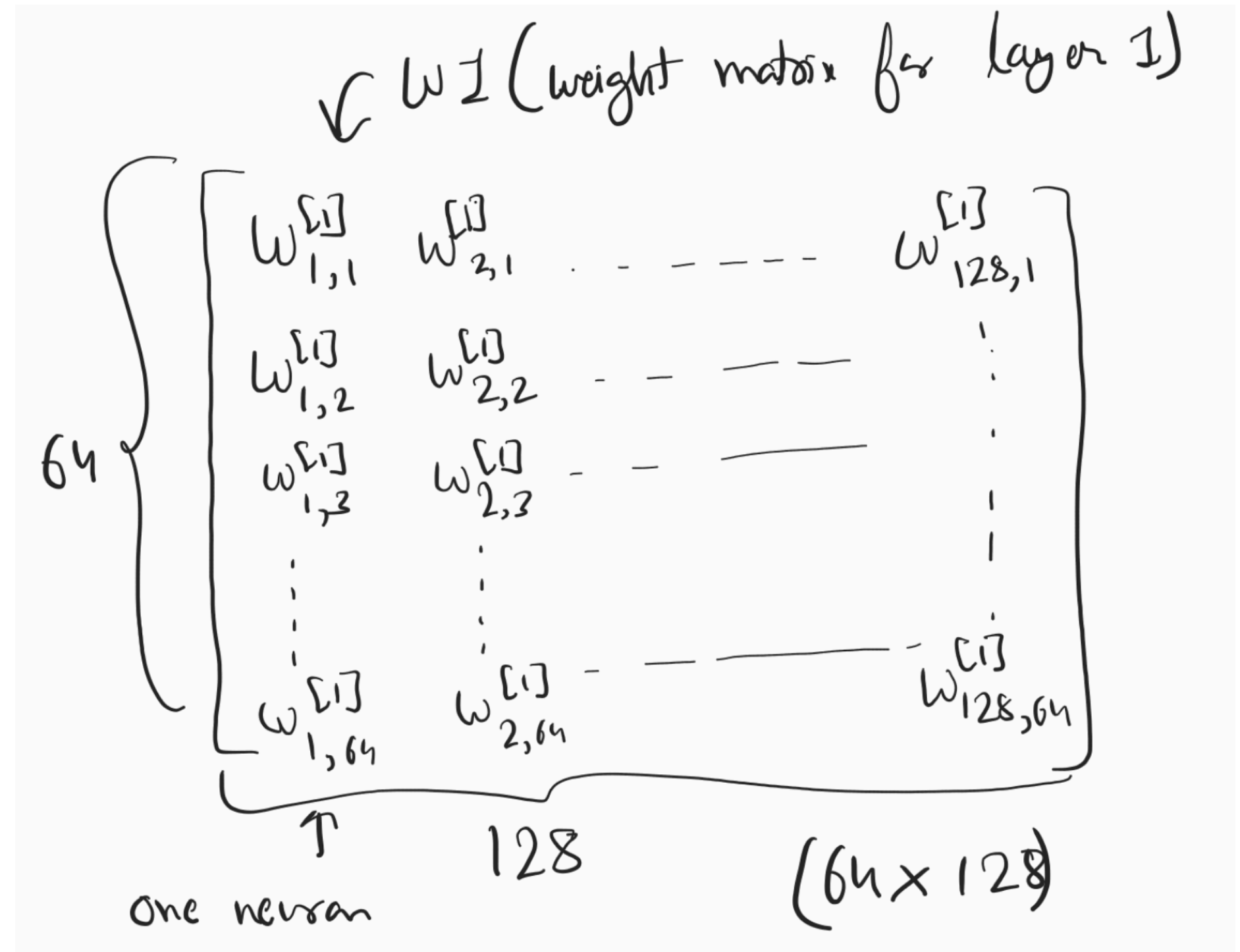
$$A1 \Rightarrow (1, 128) \rightarrow \text{for "one image"}$$

Dimension of Weight matrices

The image on the right shows the weight matrix w_1 (for layer 1)

In this matrix, each column represents weights of one neuron in layer 1. The number in superscript is the layer number and first number in subscript represents neuron number in one layer.

Dims of $w_1 \rightarrow (64, 128)$



Dimension of Bias Matrices

Dims of b1 $\rightarrow$ (1, 128)

$\swarrow$ b1 (bias matrix for layer 1)

$$\begin{bmatrix} b_1^{[1]} & b_2^{[1]} & - & - & - & - & b_{128}^{[1]} \end{bmatrix} (1, 128)$$

Following the same notation, here too each column of the matrix is the bias value of one neuron and the number in subscript is the neuron number while the number in superscript is the layer number.

Dimension of weights and bias matrix in each layer

Weights and bias matrix could be found by following the trends in the last two slides.

One observable point is that for every

1. Weight matrix $\rightarrow (n_{in}, n_{out})$

2. Bias matrix $\rightarrow (1, n_{out})$

where n_{in} = number of inputs for layer (or)
number of neurons in previous layer

n_{out} = number of outputs by layer (or)
number of neurons in current layer

$$\begin{array}{l} w1 = (64, 128) \\ b1 = (1, 128) \end{array} \left. \vphantom{\begin{array}{l} w1 \\ b1 \end{array}} \right\} \text{Layer 1}$$

$$\begin{array}{l} w2 = (128, 64) \\ b2 = (1, 64) \end{array} \left. \vphantom{\begin{array}{l} w2 \\ b2 \end{array}} \right\} \text{Layer 2}$$

$$\begin{array}{l} w3 = (64, 10) \\ b3 = (1, 10) \end{array} \left. \vphantom{\begin{array}{l} w3 \\ b3 \end{array}} \right\} \text{Layer 3}$$

Forward pass

In forward pass, we move towards output from the input.

The code below shows the implementation of forward pass. Forward pass is used to do the actual prediction by the network once you train it.

In easy words, training is actually finding the right value of weights and biases which give the expected output for an input in the neural network.

$$Z1 = X \cdot w1 + b1 \Rightarrow Z1 = (m, 128)$$

$$A1 = \text{ReLU}(Z1) \Rightarrow A1 = (m, 128)$$

$$Z2 = A1 \cdot w2 + b2 \Rightarrow Z2 = (m, 64)$$

$$A2 = \text{ReLU}(Z2) \Rightarrow A2 = (m, 64)$$

$$Z3 = A2 \cdot w3 + b3 \Rightarrow Z3 = (m, 10)$$

$$A3 = \text{Softmax}(Z3) \Rightarrow A3 = (m, 10)$$

```
#Forward pass to find the output at current weights and biases
Z1 = np.dot(X_train, w1) + b1
A1 = relu(Z1)

Z2 = np.dot(A1, w2) + b2
A2 = relu(Z2)

Z3 = np.dot(A2, w3) + b3
A3 = softmax(Z3)

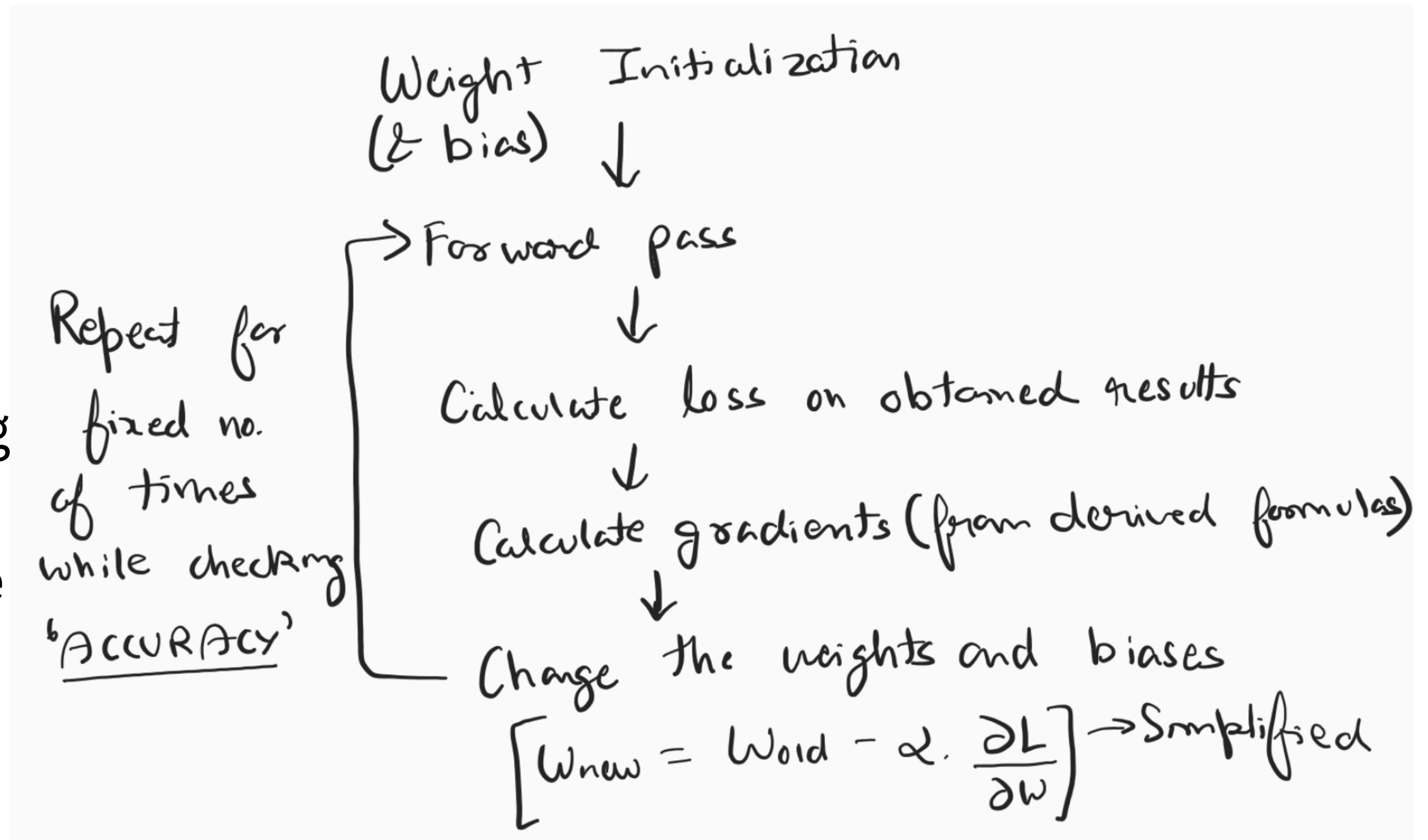
loss = crossEntropyLoss(A3, y_train)
```

Backward pass

Backward pass, as the name suggests, is moving from output towards input.

We first find the output the network is giving by doing a forward pass for initialized weights.

After that we calculate loss for that output, it is like comparing the current output with actual label to find out how far we are from real value and move towards it.



After calculating the loss, we have to determine how to change weights(and biases) to get the loss to decrease.

For this, we find the derivative of loss with respect to weights(w_1, w_2, w_3) and biases(b_1, b_2, b_3) and apply gradient descent.

By definition derivative is the change of one quantity w.r.t another and our goal it to find that wheather decreasing or increasing a particular parameter is going to increase or decrease the loss.

Then we apply standard gradient descent to change the parameters accordingly.

```
# Calculating the derivatives(gradients) with chain rule
dZ3 = (1/m) * (A3 - Y) # dZ3 = (m, 10)
dw3 = np.dot(A2.T, dZ3) # dw3 = (64, 10)
db3 = np.sum(dZ3, axis=0, keepdims=True) # db3 = (1, 10)
dA2 = np.dot(dZ3, w3.T) # dA2 = (m, 64)

dZ2 = dA2 * (Z2 > 0) # dZ2 = (m, 64)
dw2 = np.dot(A1.T, dZ2) # dw2 = (128, 64)
db2 = np.sum(dZ2, axis=0, keepdims=True) # db2 = (1, 64)
dA1 = np.dot(dZ2, w2.T) # dA1 = (m, 128)

dZ1 = dA1 * (Z1 > 0) # dZ1 = (m, 128)
dw1 = np.dot(X_train.T, dZ1) # dw1 = (64, 128)
db1 = np.sum(dZ1, axis=0, keepdims=True) # db1 = (1, 128)

# Gradient descent step of updating parameters
w1 -= learning_rate * dw1
b1 -= learning_rate * db1

w2 -= learning_rate * dw2
b2 -= learning_rate * db2

w3 -= learning_rate * dw3
b3 -= learning_rate * db3
```

Gradient Checking

The gradient checking is done by calculating gradient for a few weights and biases in two ways and then comparing their relative difference:

At first, the gradient was calculated by first principle method with the formula below.

Secondly, with the formulas for different gradients we derived, if these two give very less relative diff, of order $1e-7$, then we can assume that our gradients are correct.

$$\frac{\partial L}{\partial W} \approx \frac{L(W + \epsilon) - L(W - \epsilon)}{2\epsilon}$$

$$\text{relative diff} = \frac{|grad_{\text{analytical}} - grad_{\text{numerical}}|}{|grad_{\text{analytical}}| + |grad_{\text{numerical}}|}$$

```
--- Checking w1 (shape (64, 128)) ---
i = 5670 | analytical = -0.00708278 | numerical = -0.00708278 | rel_diff = 1.36e-09 | OK
i = 5369 | analytical = 0.00338131 | numerical = 0.00338131 | rel_diff = 2.86e-09 | OK
i = 2111 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
Max relative difference for w1: 2.86e-09

--- Checking b1 (shape (1, 128)) ---
i = 7 | analytical = 0.01698125 | numerical = 0.01698125 | rel_diff = 7.97e-10 | OK
i = 109 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
i = 8 | analytical = 0.00297100 | numerical = 0.00297100 | rel_diff = 2.26e-09 | OK
Max relative difference for b1: 2.26e-09

--- Checking w2 (shape (128, 64)) ---
i = 2241 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
i = 876 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
i = 7207 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
Max relative difference for w2: 0.00e+00

--- Checking b2 (shape (1, 64)) ---
i = 45 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
i = 49 | analytical = 0.02561313 | numerical = 0.02561313 | rel_diff = 1.79e-10 | OK
i = 7 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
Max relative difference for b2: 1.79e-10

--- Checking w3 (shape (64, 10)) ---
i = 395 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
i = 321 | analytical = 0.01480064 | numerical = 0.01480064 | rel_diff = 3.12e-10 | OK
i = 457 | analytical = 0.00000000 | numerical = 0.00000000 | rel_diff = 0.00e+00 | OK
Max relative difference for w3: 3.12e-10

--- Checking b3 (shape (1, 10)) ---
i = 0 | analytical = -0.51660731 | numerical = -0.51660731 | rel_diff = 1.32e-11 | OK
i = 8 | analytical = 0.08644310 | numerical = 0.08644310 | rel_diff = 6.92e-11 | OK
i = 5 | analytical = 0.06023919 | numerical = 0.06023919 | rel_diff = 1.49e-10 | OK
Max relative difference for b3: 1.49e-10
```

Training Loop

The training loop is nothing but combining this forward pass, loss calculation and backward pass, while keeping check on accuracy and loss.

The training loop is executed many times, predefined in the code as 'epochs'.

The learning rate is also pre-defined and set to 0.1. Changing epochs and learning rate affects the time taken and quality of training of model.

```
m = y_train.shape[0]
Y = np.zeros((m, 10))
Y[np.arange(m), y_train] = 1

for epoch in range(epochs):
    #Forward pass to find the output at current weights and biases
    Z1 = np.dot(X_train, w1) + b1
    A1 = relu(Z1)

    Z2 = np.dot(A1, w2) + b2
    A2 = relu(Z2)

    Z3 = np.dot(A2, w3) + b3
    A3 = softmax(Z3)

    loss = crossEntropyLoss(A3, y_train)
    loss_history.append(loss)
    # Calculating the derivatives(gradients) with chain rule
    dZ3 = (1/m) * (A3 - Y) # dZ3 = (m, 10)
    dw3 = np.dot(A2.T, dZ3) # dw3 = (64, 10)
    db3 = np.sum(dZ3, axis=0, keepdims=True) # db3 = (1, 10)
    dA2 = np.dot(dZ3, w3.T) # dA2 = (m, 64)

    dZ2 = dA2 * (Z2 > 0) # dZ2 = (m, 64)
    dw2 = np.dot(A1.T, dZ2) # dw2 = (128, 64)
    db2 = np.sum(dZ2, axis=0, keepdims=True) # db2 = (1, 64)
    dA1 = np.dot(dZ2, w2.T) # dA1 = (m, 128)

    dZ1 = dA1 * (Z1 > 0) # dZ1 = (m, 128)
    dw1 = np.dot(X_train.T, dZ1) # dw1 = (64, 128)
    db1 = np.sum(dZ1, axis=0, keepdims=True) # db1 = (1, 128)

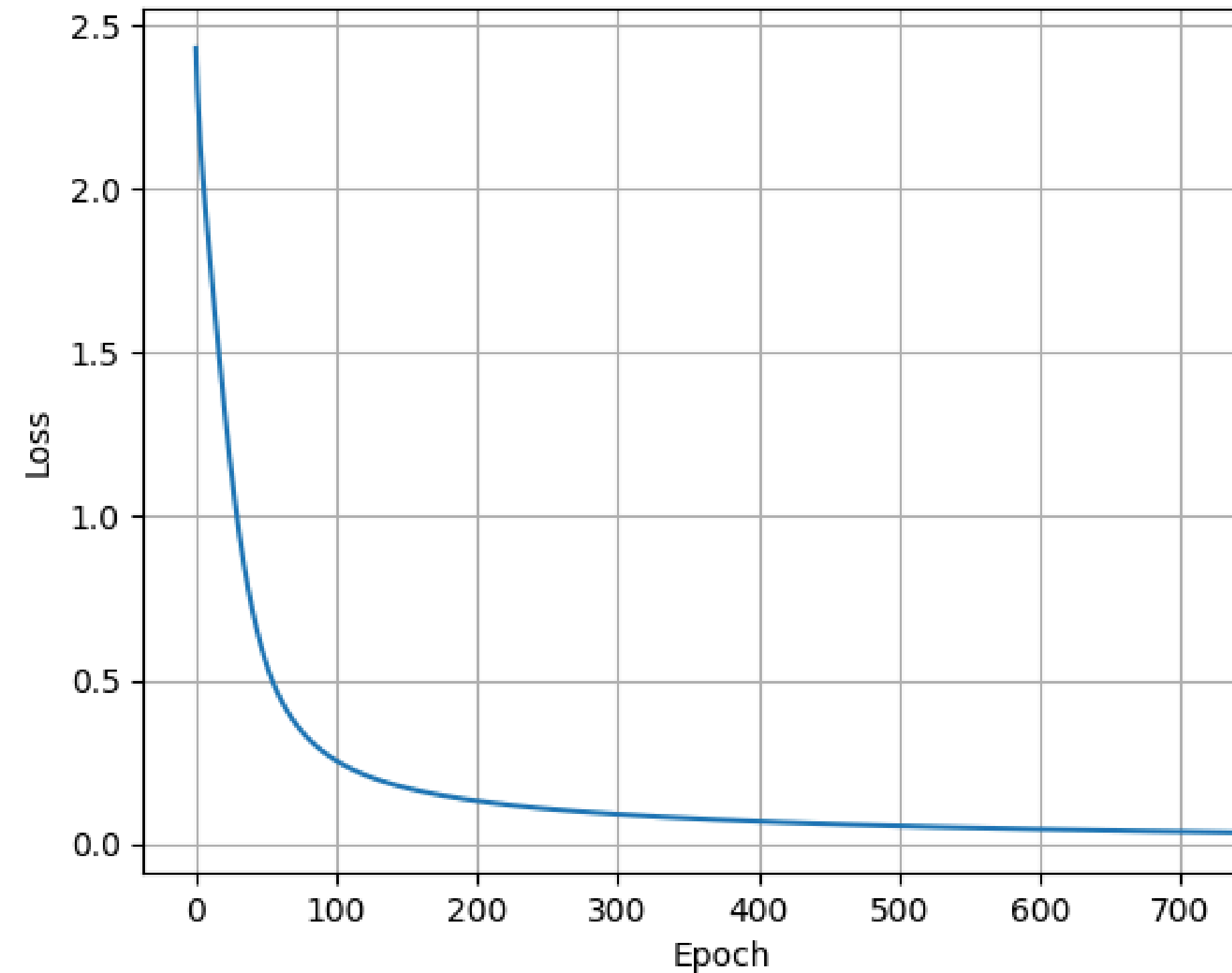
    # Gradient descent step of updating parameters
    w1 -= learning_rate * dw1
    b1 -= learning_rate * db1

    w2 -= learning_rate * dw2
    b2 -= learning_rate * db2

    w3 -= learning_rate * dw3
    b3 -= learning_rate * db3

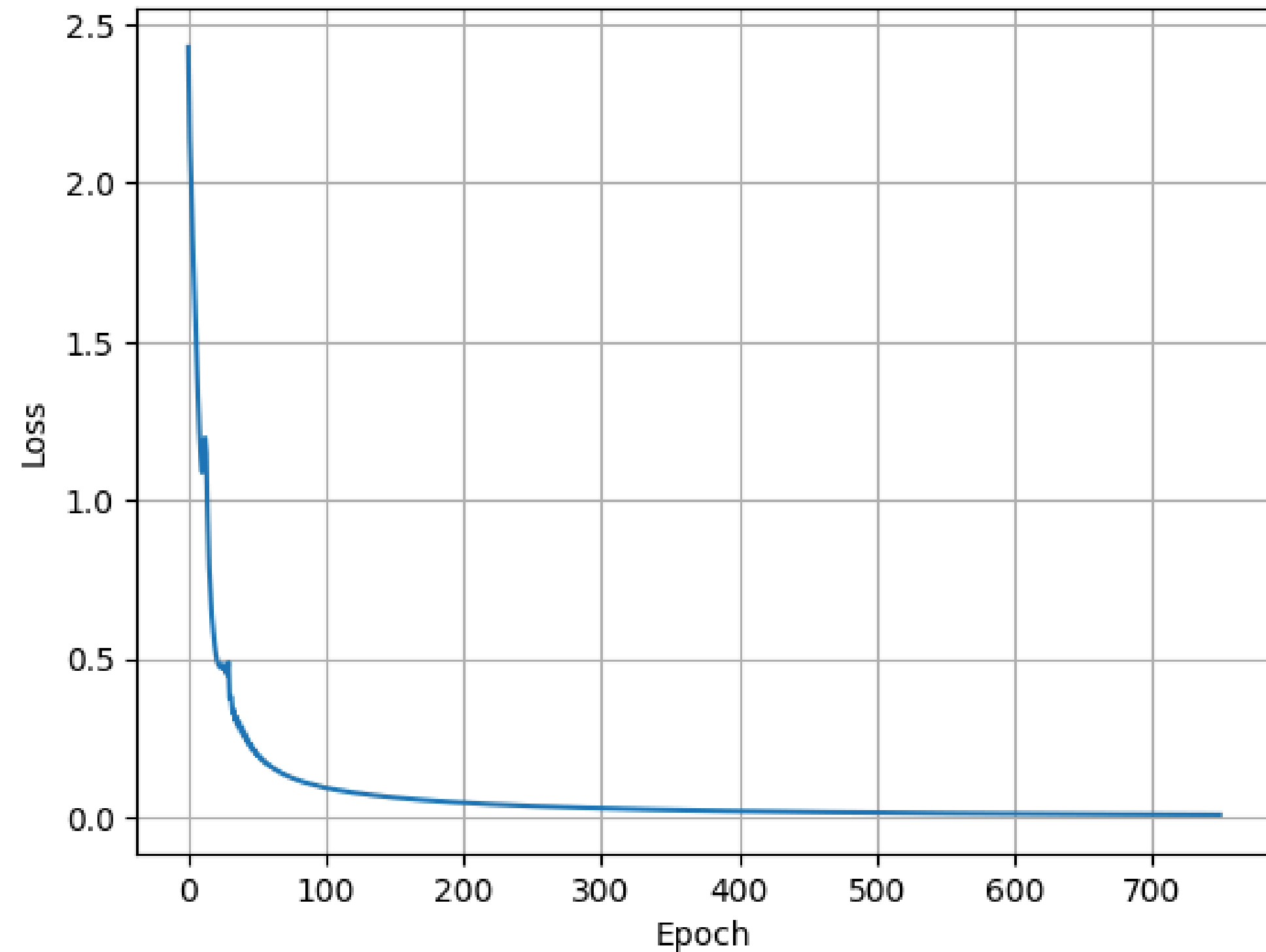
    if (epoch+1) % 50 == 0:
        preds = np.argmax(A3, axis=1)
        acc = np.mean(preds == y_train)
        print(f"Epoch: {epoch+1} | Loss: {loss:.4f} | Acc: {acc:.4f}")
        print(f"Weights: w1[5, 10]: {w1[5, 10]:.4f}, w2[1, 2]: {w2[1, 2]:.4f}, w3[1,2]: {w3[1,2]:.4f}")
        print(f"Biases: b1[0, 5]: {b1[0, 5]:.4f}, b2[0,2]: {b2[0,2]:.4f}, b3[0,9]: {b3[0,9]:.4f}")
        print("-----")
```

Loss x Epoch

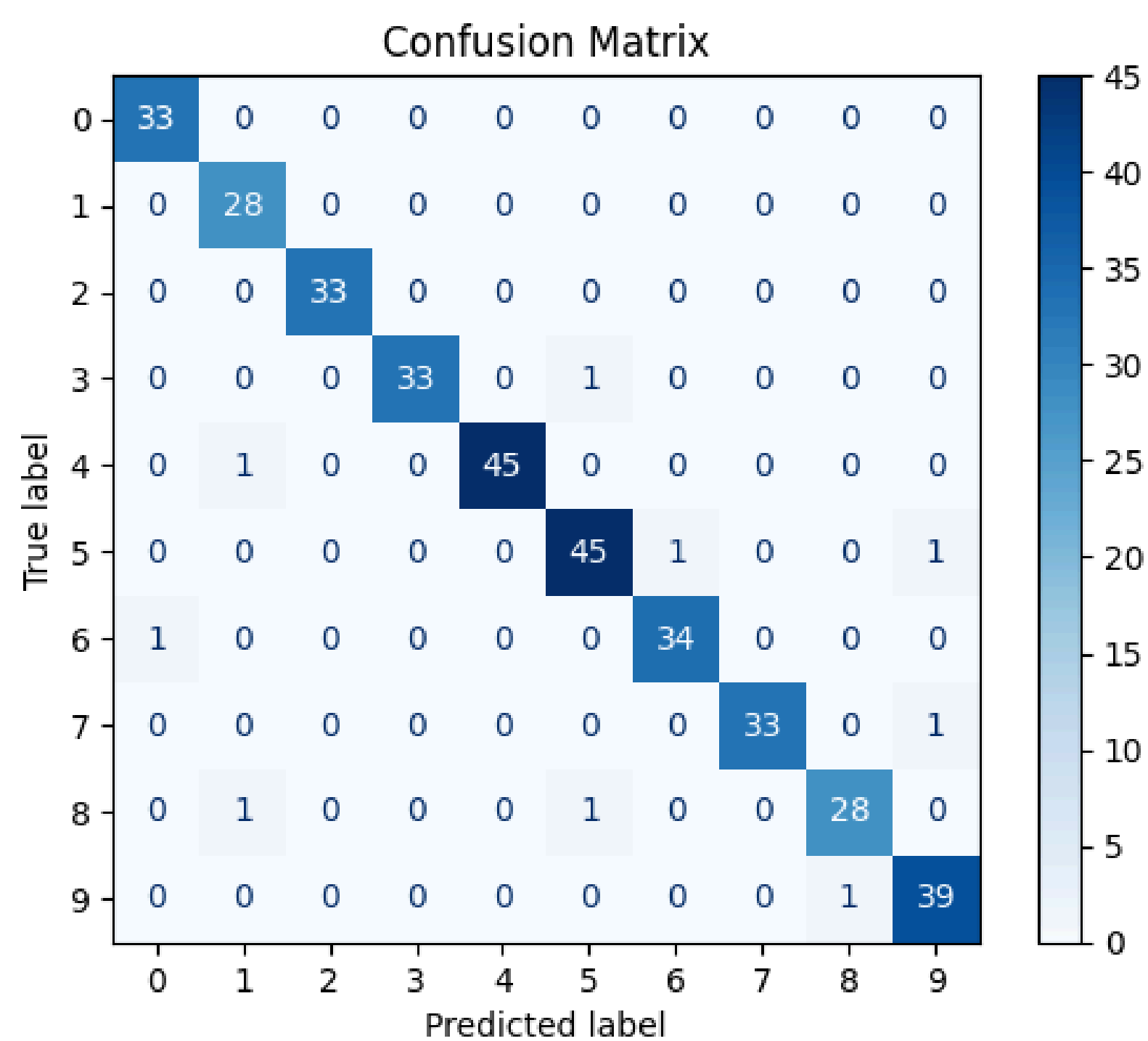


Loss curve at learning_rate = 0.1
(this one is used)
(slow convergence)

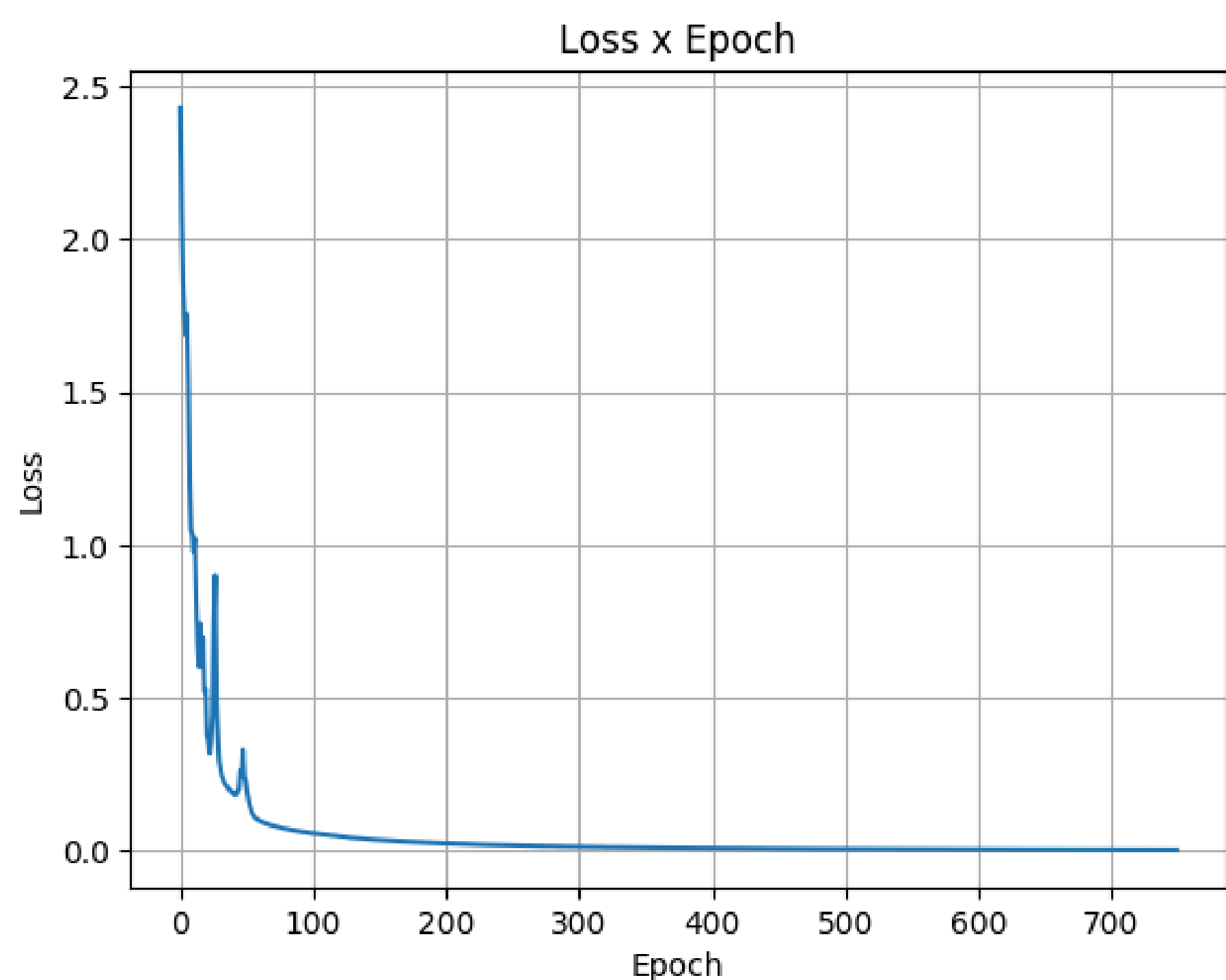
Loss x Epoch



Loss curve at learning_rate = 0.3
(moderate convergence)

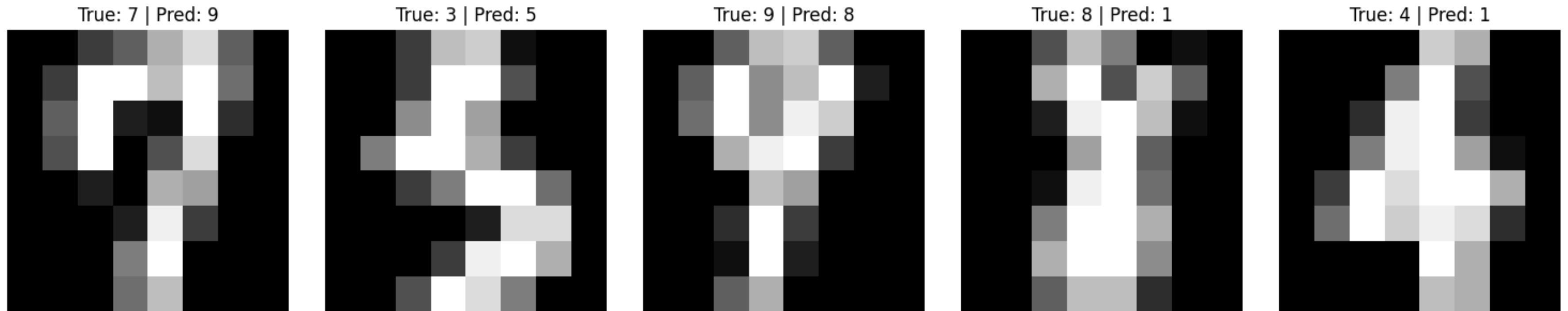


Confusion Matrix
for 97.5% accuracy
on scratch implementation



Loss curve at learning_Rate = 0.5
(faster convergence)

Misclassified images



The images that were misclassified by the model contain ambiguity as we can see in the images themselves, as a '7' with a curve that goes too down can confuse to model to assume that it's a '9'. Similar ambiguity can be seen in other images. This shows that our model is working good on normal images.

Scratch Implementaion

Max accuracy: 98.0556 %

Most of the time: ~97.5%

More lines of code

No optimization untill self-implemented from scratch

Less memory efficient.

This implementation is made for only this type of model, with crossentropy loss, relu and softmax activation.

Framework Implementation

Max accuracy: 98.33%

Most of the time: ~97.78%

2 to 3 times less line of code

Automatic optimization in many ways, in training, in data handling, in calculation, etc.

More memory efficient because of optimized code and implementation

Works for all types of activation and loss function with better options.

THANK YOU !

The framework didn't do anything conceptually different — it automated the exact same chain-rule computations, which is why doing it by hand first made the framework version make sense rather than feel like magic.

~Some LLM